

SD202 – Circuitos Digitais II

A-407 Arquiteturas para Machine Learning

Autores	
Anderson Max A e Porfirio (HBR)	Daniel Muñoz Arboleda (UnB)
Nome 3 (INSTITUIÇÃO)	Nome 4 (INSTITUIÇÃO)

Histórico de revisões		
11/08/2025	V1.0	Versão inicial

Tópicos

- Introdução
- Aprendizado Supervisionado
- Regressão Linear (Linear Regression)
- Regressão Logística (Logistic Regression)
- Árvores de Decisão (Decision Tree)
- Floresta Aleatória (Random Forest)
- Máquina de Vetores de Suporte- SVM (Support-Vector Machine)
- Aprendizado Não Supervisionado
- K-Means
- Agrupamento Espacial de Aplicativos com Ruído Baseado em Densidade - DBSCAN
- (Density-Based Spatial Clustering of Applications with Noise)
- Análise de Componentes Principais - PCA (Principal Component Analysis)
- Autoencoders
- Aprendizado por Reforço
- Aprendizagem Q (Q-Learning)
- Aprendizado Q Profundo- DQN (Deep Q-Networks)
- Gradiente de políticas (Policy Gradient)
- Hands on
- Redes Neurais

Página em branco.

Aula 12 e 13

Arquiteturas para Machine Learning (ML)

Introdução

- Definição de Machine Learning (ML): É uma área da Inteligência Artificial em que se ensina a máquina, com dados históricos, a resolver problemas e melhorar o desempenho de algumas tarefas, sem serem explicitamente programadas para isso.
- A máquina aprende a realizar a tarefa.
- Veremos três tipos de aprendizado
 - Aprendizado Supervisionado
 - Aprendizado Não Supervisionado
 - Aprendizado Por Reforço

Introdução

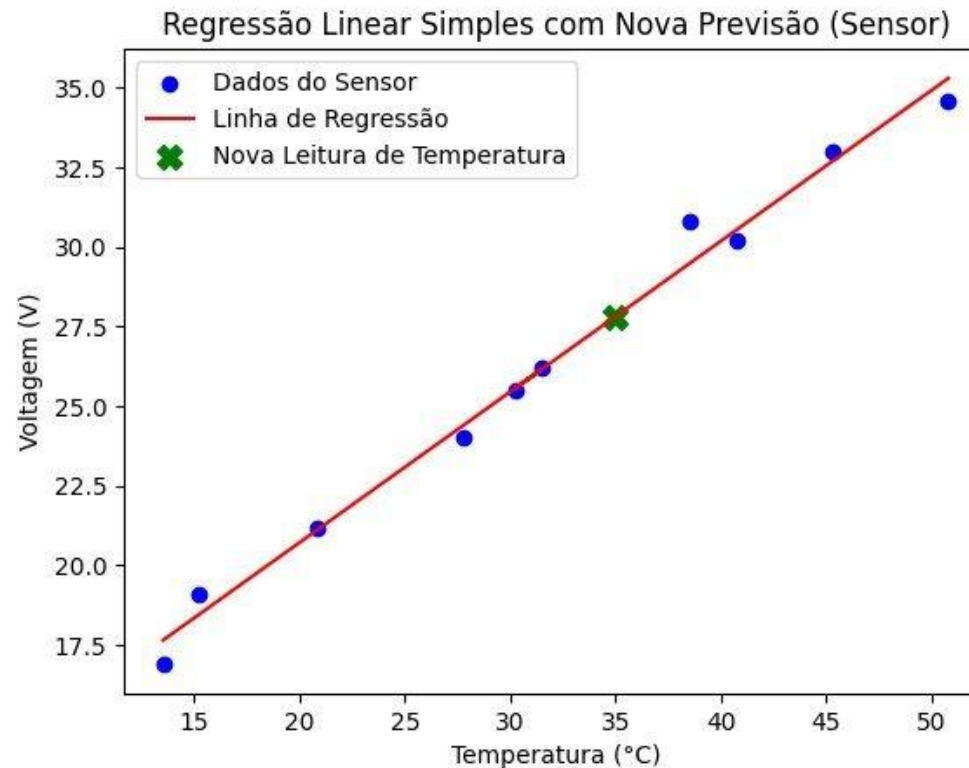
- Passos
 - Coleta de Dados: Pegar dados relevantes
 - Pré-processamento: Limpar dados (tirar valores faltantes, padronizar)
 - Treinamento: O modelo "estuda" os dados
 - Avaliação: Avaliamos parâmetros e testamos o modelo com dados novos
 - Utilização: Usar o modelo treinado
- Motivação para integrar Machine Learning com Hardware: Velocidade, independência, eficiência, operações off-line entre outros benefícios
 - Dispositivos Médicos
 - Marcapassos que detectam arritmias e agem sem depender de conexão
 - Agricultura de Precisão
 - Sensores no solo + ML embarcado = irrigação automática baseada em umidade
 - Indústria 4.0
 - Robôs que identificam defeitos em peças na linha de produção

Aprendizado Supervisionado

- O algoritmo é treinado usando um conjunto de dados rotulados, onde cada exemplo de entrada possui a saída correta correspondente.
- Alguns tipos de aprendizado supervisionado:
 - Regressão Linear
 - Regressão Logística
 - Árvores de Decisão
 - Floresta Aleatória
 - Support Vector Machines
 - Redes Neurais*

Aprendizado Supervisionado

- Regressão Linear tenta encontrar a função (reta, plano ou hiperplano) que melhor se ajusta a um treinamento, para realizar uma previsão.
 - $y = w \cdot x + b$ (w e b são obtidos no treinamento)



Aprendizado Supervisionado

```
import numpy as np
from sklearn.linear_model import LinearRegression
import matplotlib.pyplot as plt

# Dados de exemplo simulando um sensor
X = np.array([[13.52810469], [15.24475886], [20.84636486], [27.81511973], [31.51289376], [30.26766646], [38.5668435 ], [40.80839669], [45.34911785], [50.821197 ]]) #
Temperatura com ruído
y = np.array([16.90809592], [19.07665294], [21.18422015], [24.02923488], [26.20031011], [25.46750756], [30.77750082], [30.19904008], [32.98762663], [34.55650276])) #
Voltagem com ruído e relação linear

# Criar e treinar o modelo de regressão linear
model = LinearRegression()
model.fit(X, y)

# Obter os parâmetros do modelo
r_sq = model.score(X, y)
intercept = model.intercept_
coef = model.coef_

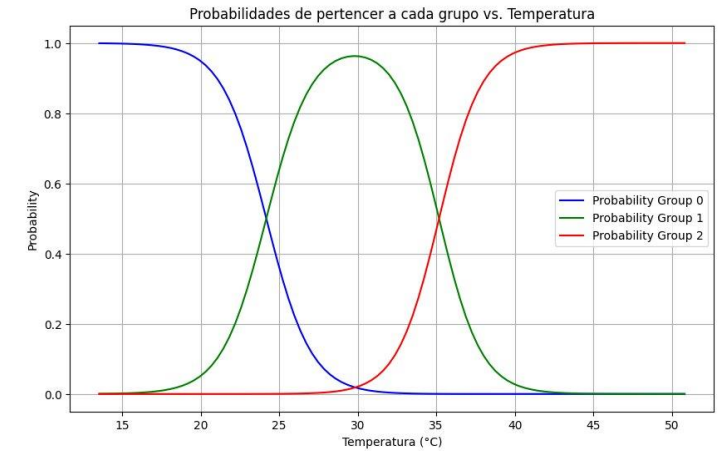
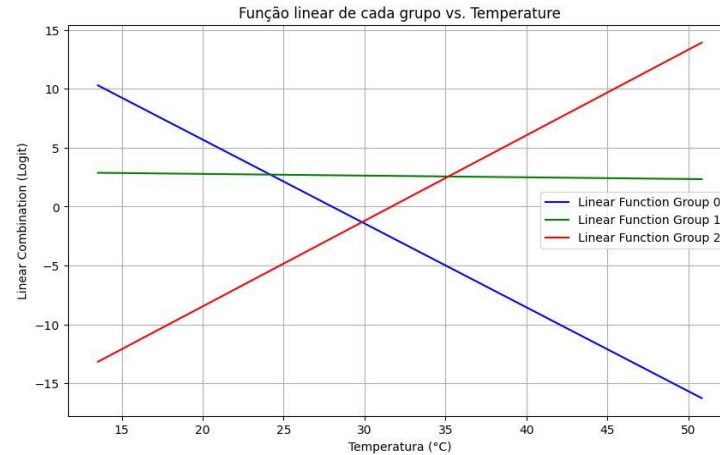
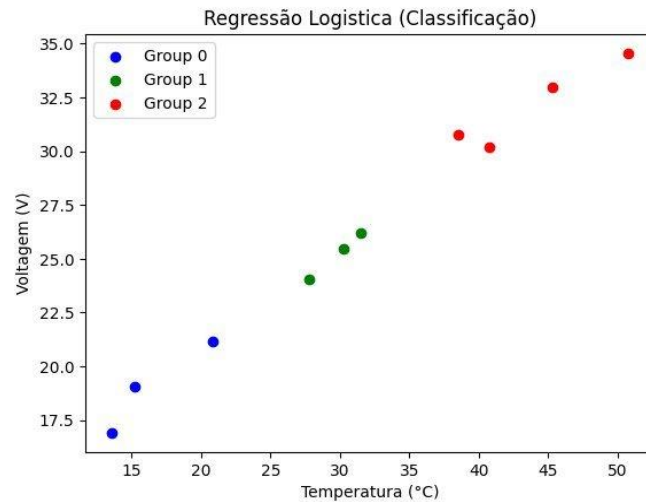
# Qualidade do modelo dos dados treinados. Quanto R2 mais próximo de 1, melhor o modelo.
print(f"Coeficiente de determinação (R2) do modelo do sensor: {r_sq}")
print(f"Intercepção (Voltagem base): {intercept}")
print(f"Coeficiente (Variação da Voltagem por Temperatura): {coef}")

>>>>>>>> OUTPUT
Coeficiente de determinação (R2) do modelo do sensor: 0.9891260349411788
Intercepção (Voltagem base): [11.25802731]
Coeficiente (Variação da Voltagem por Temperatura): [[0.47276083]]
```

Aprendizado Supervisionado

- Regressão Logística é usada na classificação binária, ordinal ou multinomial.
 - $y = w \cdot x + b$ (reta em 2D)
 - $y = w_1 \cdot x_1 + w_2 \cdot x_2 + \dots + w_n \cdot x_n + b$ (hiperplano)
 - $\sigma(y) = 1 / (1 + e^{-y})$ vai resultar na probabilidade de pertencer ao grupo
 - e: base do logaritmo natural
 - b: intercepto do modelo
 - w: coeficiente do atributo
 - A função sigmoide transforma o valor numérico em uma probabilidade entre 0 e 1
 - Multinomial - Um único modelo calcula simultaneamente as probabilidades para todas as classes. A fórmula é baseada em funções softmax, que comparam os “escores” de cada classe para gerar probabilidades que somam 1
- Uso típico em hardware:
 - Sensores inteligentes que fazem previsão simples (ex.: sensores industriais)
 - Sistemas embarcados com recursos limitados

Aprendizado Supervisionado



>>>>>>> OUTPUT

Weights (w): -0.71168225 / -0.01440386 / 0.72608611

Biases (b): 19.91985936 / 3.06481866 / -22.98467801

$\sigma(y) = 1 / (1 + e^{-(wx + b)})$

Temperaturas (°C): [10 25 40 55]

index	Probability Group 0	Probability Group 1	Probability Group 2
0	0.9999489297229458	5.1070276646007904e-05	4.082466680334153e-13
1	0.35951916769322023	0.6401397259081648	0.0003411063986149041
2	4.411144338473601e-07	0.02738207388435816	0.972617485001208
3	1.9515785002562414e-16	4.223411845479706e-07	0.9999995776588152

Aprendizado Supervisionado

```
import numpy as np
from sklearn.linear_model import LogisticRegression

X = np.array([[13.52810469], [15.24475886], [20.84636486], [27.81511973], [31.51289376], [30.26766646], [38.5668435 ], [40.80839669], [45.34911785], [50.821197
]]) # Temperatura
y = np.array([16.90809592, 19.07665294, 21.18422015, 24.02923488, 26.20031011, 25.46750756, 30.77750082, 30.19904008, 32.98762663, 34.55650276]) #
    Voltagem

threshold_low = np.percentile(y_flat, 33.3)
threshold_medium = np.percentile(y_flat, 66.6)
print(f"\nThreshold para o primeiro grupo: {threshold_low}")
print(f"Threshold para o grupo do meio: {threshold_medium}")

model_lr = LogisticRegression()

# Treinando o modelo com os dados
model_lr.fit(X, y_categorical.flatten())

weights = model_lr.coef_
biases = model_lr.intercept_
print("\n(w) de cada grupo:\n", weights)
print("(b) de cada grupo:\n", biases)

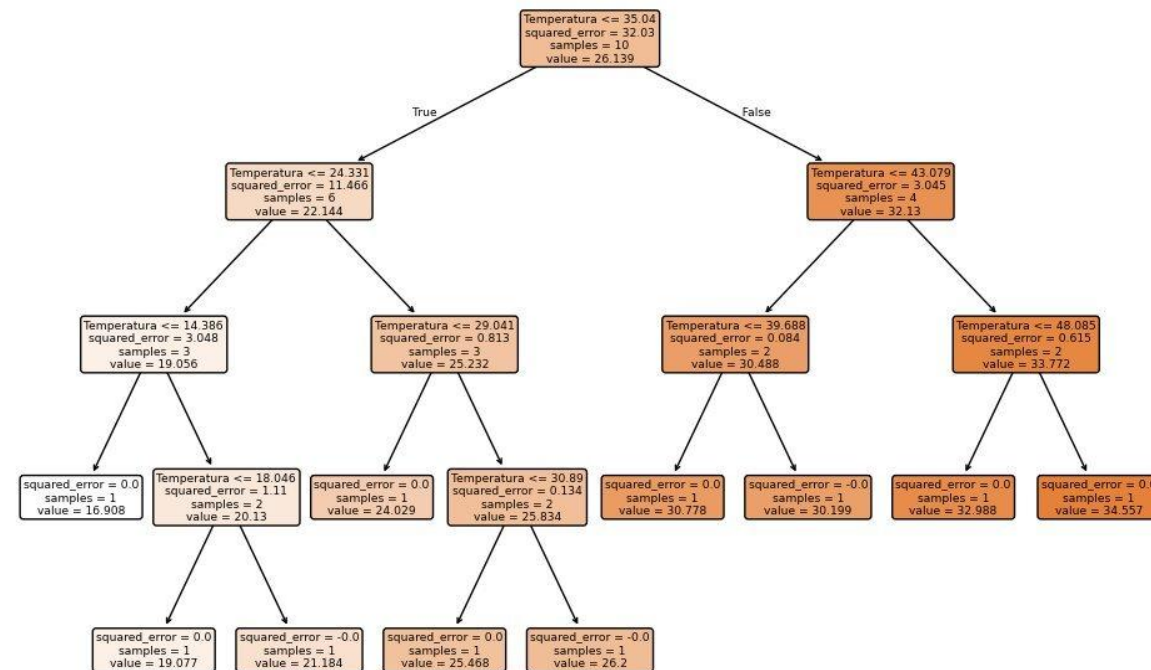
>>>>>>>OUTPUT

Coeficiente de determinação ( $R^2$ ) do modelo do sensor: 0.9891260349411788
Intercepção (Voltagem base): [11.25802731]
Coeficiente (Variação da Voltagem por Temperatura): [[0.47276083]]
```

Aprendizado Supervisionado

- Árvores de Decisão é baseada em comparações condicionais (if/else) que levam a uma predição
 - Um conjunto de dados treina o modelo, que nos fornece a árvore

Árvore de Decisão para Regressão (Temperatura vs Voltagem)



Aprendizado Supervisionado

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.tree import DecisionTreeRegressor, plot_tree

# Data fornecida
X = np.array([[13.52810469], [15.24475886], [20.84636486], [27.81511973], [31.51289376], [30.26766646], [38.5668435 ],
[40.80839669], [45.34911785], [50.821197 ]]) # Temperatura
y = np.array([[16.90809592], [19.07665294], [21.18422015], [24.02923488], [26.20031011], [25.46750756], [30.77750082],
[30.19904008], [32.98762663], [34.55650276]]) # Voltagem

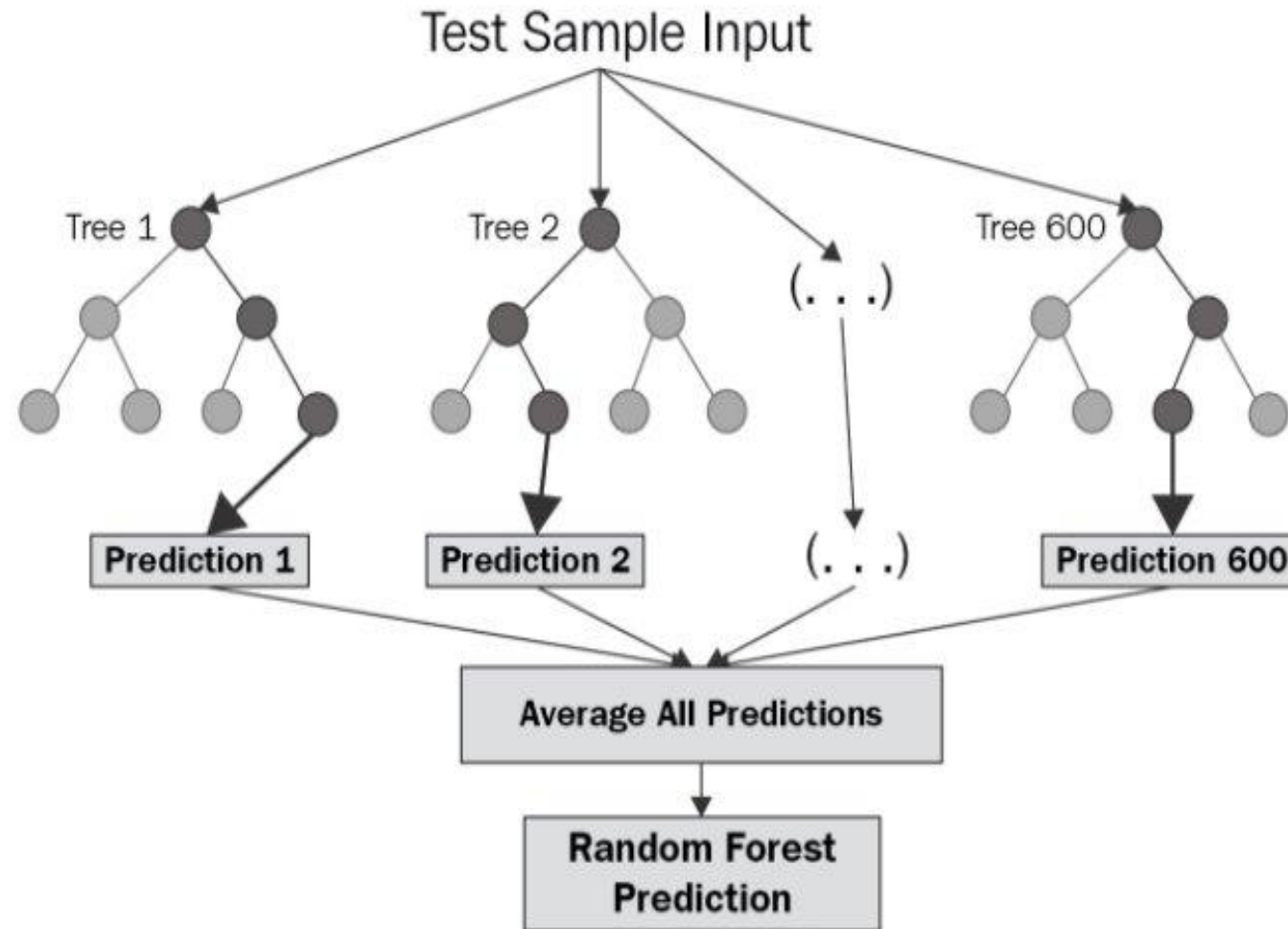
# Instanciar e treinar o modelo de árvore de decisão para regressão
regressor = DecisionTreeRegressor(random_state=42)
regressor.fit(X, y)

# Visualizar a árvore
plt.figure(figsize=(12, 8))
plot_tree(regressor, feature_names=['Temperatura'], filled=True, rounded=True)
plt.title("Árvore de Decisão para Regressão (Temperatura vs Voltagem)")
plt.show()
```

Aprendizado Supervisionado

- Floresta Aleatória (Random Forest)
 - Conjunto de árvores de decisão, o que torna bem mais pesado para rodar uma predição
 - Subconjuntos dos dados treinam as árvores
 - Cada árvore é independente, o que permite paralelização em hardware
 - Os resultados precisam ser processados
 - § Podendo usar uma média dos vários outputs para decidir a predição
 - § Ou uma votação onde a maioria vence
- Uso típico em hardware:
 - Sistemas embarcados para classificação rápida (ex.: detecção de falhas)
 - FPGAs podem executar múltiplas árvores em paralelo

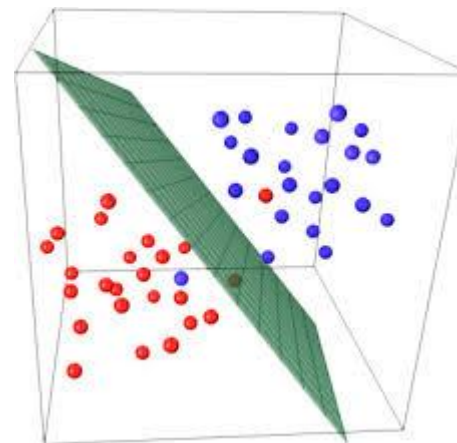
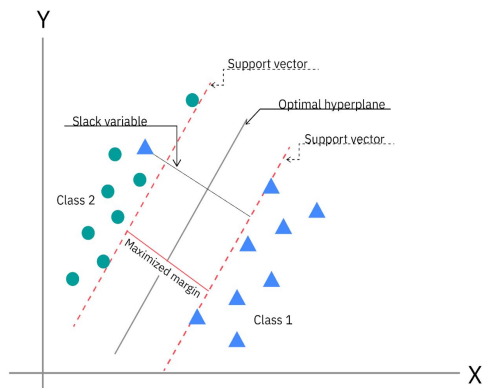
Aprendizado Supervisionado



Aprendizado Supervisionado

- SVM (Support Vector Machines)
 - Busca encontrar uma fronteira de decisão que separa classes de dados no espaço de atributos.
 - Cria a maior margem possível entre as classes.
 - Essa margem é determinada pelos support vectors, que são os pontos mais próximos dessa fronteira.
 - Ele desconsidera outliers ao definir a melhor separação.
- Uso típico em hardware:
 - Sensores de temperatura, pressão, vibração e etc podem capturar um comportamento variável ao longo da vida de um motor.
 - Esses dados podem treinar um ML e gerar uma função que pode gerar alertas diversos e tomar decisões no ASIC.

Aprendizado Supervisionado



- Após treinar o SVM (hipotético), ele poderia encontrar a seguinte função:
- $Y = 0.02 x_1 + 0.5 x_2 - 0.8 x_3 + 1.2$
- $w_1=0.02 \rightarrow$ tem pouca influência.
- $w_2=0.5 \rightarrow$ influência mais.
- $w_3= -0.8 \rightarrow$ é o fator mais importante na decisão do grupo.
- $b=1.2 \rightarrow$ desloca o plano para melhor separação.

Aprendizado Supervisionado

- Redes Neurais* é o método mais abrangente. Imita o sistema de aprendizado e decisões do ser humano.
 - Redes neurais são usadas quando a tarefa é complexa e requer a capacidade de aprender padrões não lineares em dados
 - É inspirado na estrutura e função do cérebro humano
 - Consistem em camadas de nós interconectados (neurônios) que processam e transmitem informações
 - As redes neurais ajustam os pesos nas conexões entre os neurônios durante o treinamento para melhorar o desempenho em uma determinada tarefa

Aprendizado Supervisionado

Algoritmo	Fácil de Implementar	Bom para Hardware de Baixa Potência	Chips Dedicados Disponíveis
Regressão Linear	✅ Sim	✅ Sim	Não necessário
Regressão Logística	✅ Sim	✅ Sim	Não necessário
Árvores / RandomForest	✅ Sim	⚠️ Médio (se muitas árvores)	Não específico
Redes Neurais	⚠️ Médio / Difícil	⚠️ Médio (TinyML)	✅ GPUs, TPUs, NPUs
SVM	✅ Sim (Inferência)	✅ Sim	Não específico

- $y = w * x + b$
- $\sigma(y) = 1 / (1 + e^{-wx + b})$
- Árvore de if/else

Aprendizado Supervisionado

- Redes Neurais* é o método mais abrangente. Imita o sistema de aprendizado e decisões do ser humano.
 - Redes neurais são usadas quando a tarefa é complexa e requer a capacidade de aprender padrões não lineares em dados
 - É inspirado na estrutura e função do cérebro humano
 - Consistem em camadas de nós interconectados (neurônios) que processam e transmitem informações
 - As redes neurais ajustam os pesos nas conexões entre os neurônios durante o treinamento para melhorar o desempenho em uma determinada tarefa

Aprendizado Não Supervisionado

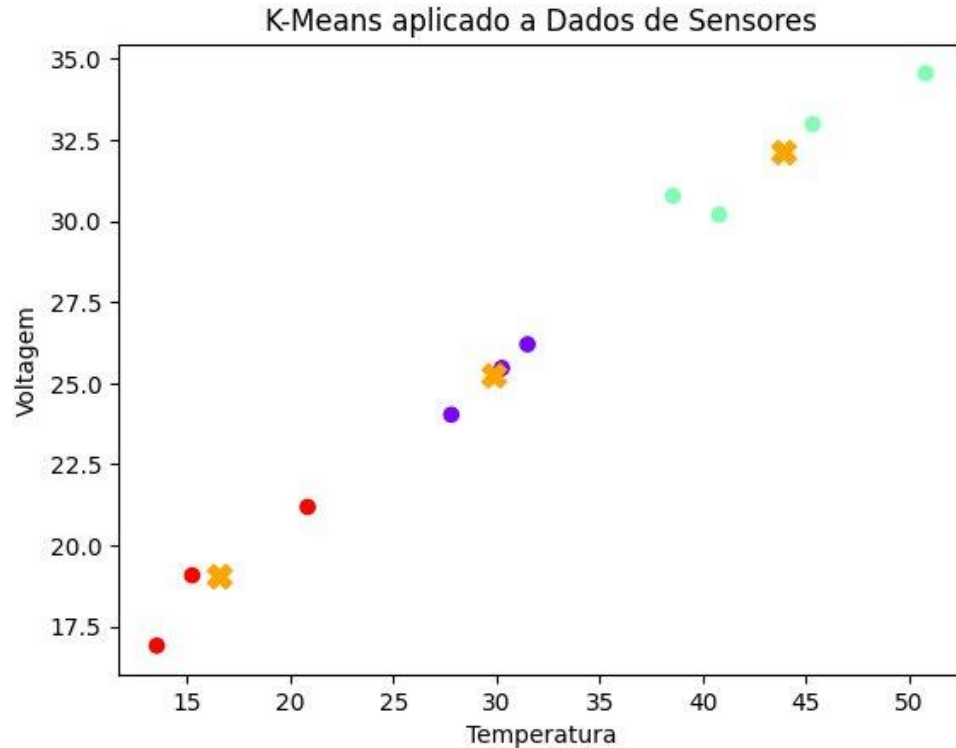
- O algoritmo é treinado usando um conjunto de dados não rotulados, onde cada exemplo de entrada não possui uma saída correspondente.
- Alguns tipos de aprendizado não supervisionado:
 - K-Means (clustering)
 - DBSCAN (Density-Based Spatial Clustering of Applications with Noise)
 - PCA (Principal Component Analysis)
 - Autoencoders

Aprendizado Não Supervisionado

- K-Means (clustering)
 - É um algoritmo não supervisionado, usado para agrupar dados similares em clusters (grupos).
 - Divide os dados em K grupos, onde cada ponto pertence ao grupo mais próximo da sua média (centroide).
 - Precisa definir quantos grupos deverão ser formados com os dados de entrada.
- Para classificar um novo ponto, calcula-se a distância entre esse ponto e cada centroide, atribuindo-o ao grupo cujo centroide estiver mais próximo.

$$\text{Distância} = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

Aprendizado Não Supervisionado



$$\text{Dist_g0} = \sqrt{(16.53 - x_n)^2 + (19.05 - y_n)^2}$$

$$\text{Dist_g1} = \sqrt{(43.88 - x_n)^2 + (32.13 - y_n)^2}$$

$$\text{Dist_g2} = \sqrt{(29.86 - x_n)^2 + (25.23 - y_n)^2}$$

A distância entre o ponto novo e o centro da esfera é $d = \sqrt{(x_c - x_1)^2 + (y_c - y_1)^2 + (z_c - z_1)^2}$

Aprendizado Supervisionado

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans

# Dados de exemplo simulando um sensor
x = np.array([[13.52810469], [15.24475886], [20.84636486], [27.81511973], [31.51289376], [30.26766646],
[38.5668435 ], [40.80839669], [45.34911785], [50.821197 ]]) # Temperatura com ruído
y = np.array([[16.90809592], [19.07665294], [21.18422015], [24.02923488], [26.20031011], [25.46750756],
[30.77750082], [30.19904008], [32.98762663], [34.55650276]]) # Voltagem com ruído e relação linear

# Combinar x e y em um único array de pares
X = np.hstack((x, y))

# Aplicar K-Means
kmeans = KMeans(n_clusters=3)
kmeans.fit(X)

print("Os valores dos centroides são:")
print(kmeans.cluster_centers_)

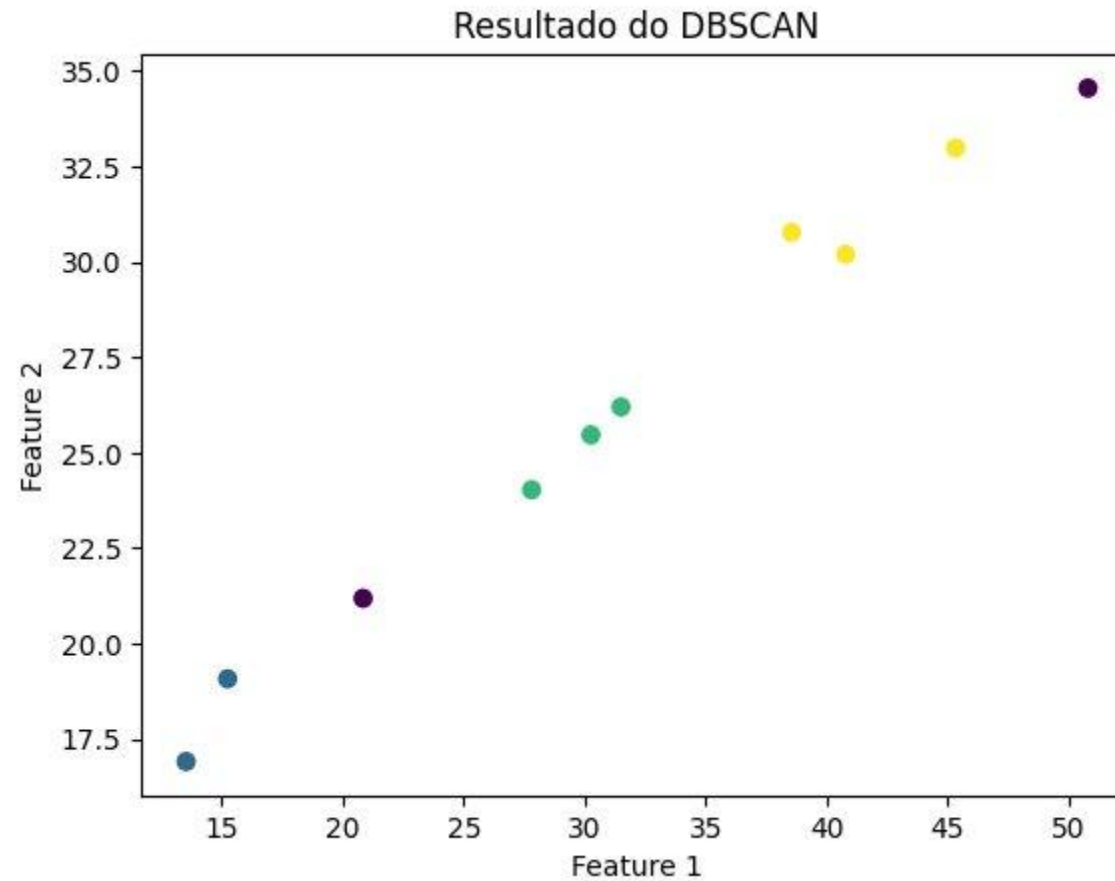
>>>>>>>>OUTPUT
Os valores dos centroides são: [[16.5397428 19.056323 ] [43.88638876 32.13016757] [29.86522665 25.23235085]]
```

Aprendizado Não Supervisionado

- DBSCAN (Density-Based Spatial Clustering of Applications with Noise)
 - Identifica os agrupamentos densos de dados (clusters)
 - Ignora os dados solitários (ruído/outliers)
 - Não precisa saber quantos grupos existem antes de começar
 - Precisa do raio de alcance
- Uso típico em hardware
 - Identificar padrões de falha
 - Sensores defeituosos
- O DBSCAN é uma ferramenta poderosa para a análise de dados de sensores, pois ajuda a encontrar padrões escondidos e anomalias sem precisar saber o que está procurando antecipadamente.

Aprendizado Não Supervisionado

Eps = 5.5 (raio)
min_samples = 2



Aprendizado Supervisionado

```
import numpy as np
from sklearn.cluster import DBSCAN
import matplotlib.pyplot as plt

# Dados fornecidos
X = np.array([[13.52810469, 16.90809592],[15.24475886, 19.07665294],[20.84636486, 21.18422015],
[27.81511973, 24.02923488],[31.51289376, 26.20031011],[30.26766646, 25.46750756],
[38.5668435, 30.77750082],[40.80839669, 30.19904008],[45.34911785, 32.98762663],[50.821197, 34.55650276]])

# epsilon (eps): A distância máxima entre duas amostras para que uma seja considerada na vizinhança da outra.
# min_samples: O número de amostras em uma vizinhança para um ponto ser considerado um ponto central.
dbscan = DBSCAN(eps=5.5, min_samples=2)
clusters = dbscan.fit_predict(X)

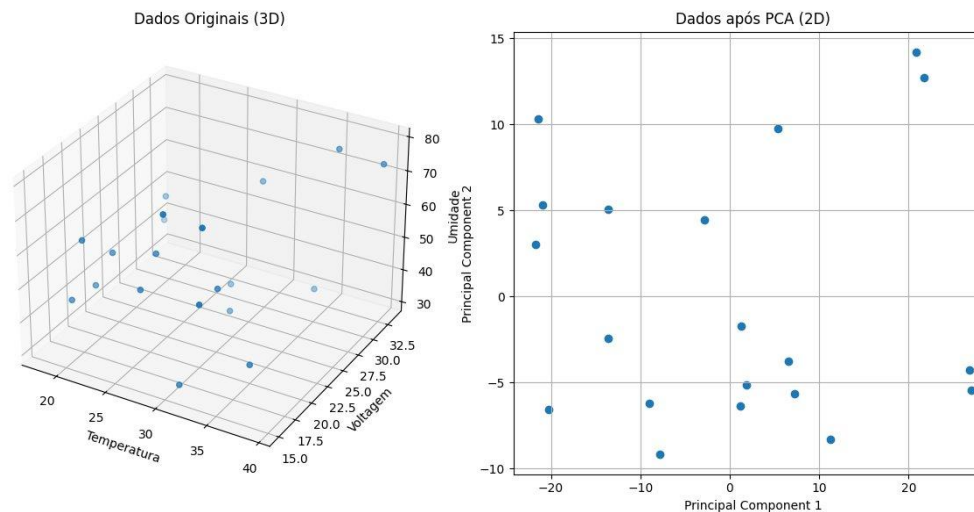
# Exibindo os resultados
print("Rótulos dos clusters:", clusters)

# Visualizando os resultados
plt.scatter(X[:, 0], X[:, 1], c=clusters, cmap='viridis')
plt.title('Resultado do DBSCAN')
plt.xlabel('Feature 1')
plt.ylabel('Feature 2')
plt.show()
```

Aprendizado Não Supervisionado

- PCA (Principal Component Analysis)

- PCA é uma técnica para reduzir a dimensionalidade dos dados, transformando várias variáveis correlacionadas em um conjunto menor de variáveis não correlacionadas chamadas componentes principais.
- Se temos um conjunto de dados de 5 sensores, e aplicamos o PCA nesses dados, podemos identificar um novo conjunto de dados, 2D, definem 95% dos dados.
- Reduzir consumo de energia transmitindo menos dados.
- Identificar quais sensores são redundantes.
- Detectar anomalias (pontos fora do padrão nos componentes).



Aprendizado Supervisionado

```
import numpy as np
from sklearn.decomposition import PCA
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
# Gerando um conjunto de dados 3D de exemplo
X_3d = np.array([[27.6542, 33.2834, 58.5754], [33.4134, 15.0000, 58.5700], [19.1473, 30.8107, 51.0570], [33.0634, 15.9728, 78.3602], [36.1560, 18.1738, 37.4905], [21.5438, 19.7098, 54.7878], [26.1268, 19.4681, 58.9376], [19.9020, 19.4753, 43.9472], [26.7144, 29.7176, 31.9343], [28.5839, 26.7332, 30.6389], [30.9105, 15.7479, 31.1168], [39.4542, 33.4978, 72.8729], [21.0948, 16.2482, 64.0815], [26.4402, 16.7832, 53.3288], [18.0000, 32.1505, 40.4355], [31.9127, 19.8475, 53.0076], [29.3520, 15.9705, 78.9980], [35.1918, 33.3404, 74.1968], [33.3851, 32.7671, 30.8645], [19.7636, 16.3840, 44.9736]])

# Inicializa o modelo PCA para reduzir para 2 componentes principais
pca = PCA(n_components=2)

# Ajusta o modelo aos dados 3D e transforma os dados para 2D
X_pca_2d = pca.fit_transform(X_3d)
print("Formato dos dados após PCA (2D):", X_pca_2d.shape)
print("\nDados após PCA (2D):")
print(X_pca_2d)

>>>>>>>>OUTPUT
Formato dos dados após PCA (2D): (20, 2)
Dados após PCA (2D)

[[ 5.39077328  9.78910609] [ 7.25917921 -5.65737517] [-2.81751058  4.48369286] [ 26.78206909 -4.2538406 ] [-13.56697041 -2.4055295 ] [ 1.90114804 -5.13663484] [ 6.52942662
-3.76467045] [-9.01190845 -6.21352353] [-20.88892053  5.31037505] [-21.76719386  3.04936012] [-20.29013209 -6.58200593] [ 20.83326992 14.22204772] [ 11.30647131
-8.27702932] [ 1.184329 -6.37319678] [-13.56740144  5.06232684] [ 1.24861256 -1.7210605 ] [ 27.01301937 -5.43191605] [ 21.69563401 12.74043425] [-21.43635845 10.31180177]
[ -7.79753661 -9.15236203]]
```

Aprendizado Não Supervisionado

- Autoencoders
 - Utiliza redes neurais apenas com dados de entrada no treinamento, sem fornecer os rótulos.
 - Aprende a comprimir (codificar) dados e depois a recriar (decodificar) a versão aproximada da versão original a partir dessa forma comprimida.
 - Encoder reduz a dimensionalidade dos dados (como compactar um arquivo ZIP).
 - O bottleneck (gargalo) força a rede a aprender apenas as características mais importantes dos dados.
 - Decoder reconstrói os dados da melhor forma a partir da versão compactada.

Aprendizado Não Supervisionado

Algoritmo	Fácil de Implementar	Bom para Hardware de Baixa Potência	Chips Dedicados Disponíveis
K-Means (clustering)	✅ Sim	✅ Sim	Não específico
DBSCAN	⚠️ Médio	❌ Não (custo alto de distâncias)	Não específico
PCA	⚠️ Médio	⚠️ Médio (depende da dimensão)	Não específico
Autoencoders	⚠️ Médio / Difícil	⚠️ Médio (TinyML)	✅ GPUs, TPUs, NPUs

K-Means pode classificar novos dados segundo um modelo.

DBSCAN identifica agrupamentos.

PCA e Autoencoders irão reduzir a quantidade de dados.

Aprendizado Por Reforço

- Essencialmente, o agente experimenta o ambiente, recebe feedback na forma de recompensas ou penalidades e, com base nesse feedback, aprende a tomar decisões ótimas.
 - Q-Learning
 - Deep Q-Networks (DQN)
 - Policy Gradient

Aprendizado Por Reforço

- Q-Learning
 - Ensina um "agente" a tomar as melhores decisões em um ambiente para maximizar uma recompensa cumulativa ao longo do tempo.
 - Aprende a agir sem precisar de um modelo pré-existente de como o ambiente funciona, por tentativa e erro, interagindo com o ambiente.
 - Constrói uma tabela, chamada Q-Table, que armazena a "qualidade" de uma determinada ação em um determinado estado.
 - Ao longo do tempo, essa tabela contém os valores ideais que dirão ao agente qual é a melhor ação a ser tomada em cada situação.
 - O Q-Learning é especialmente útil para problemas com um número discreto e finito de estados e ações, como jogos de tabuleiro, navegação em labirintos ou robôs simples.
 - Ambientes com um número muito grande de estados, a Q-Table se torna impraticavelmente grande.

Aprendizado Por Reforço

- Deep Q-Learning
 - É uma variação mais avançada do Q-Learning.
 - Ensina um "agente" a tomar as melhores decisões em um ambiente para maximizar uma recompensa cumulativa ao longo do tempo.
 - Aprende a agir sem precisar de um modelo pré-existente de como o ambiente funciona, por tentativa e erro, interagindo com o ambiente.
 - Utiliza redes neurais para estimar os valores Q, em vez de armazená-los em uma tabela.
 - Em vez de anotar todas as situações, o robô tem um "cérebro" que generaliza o que fazer em qualquer situação.

Aprendizado Por Reforço

- Preparar a "memória" do agente, que será usada para armazenar o que ele aprende.
 - Q-table: É uma tabela que mapeia cada par (estado, ação) para um valor de "qualidade" ou Q-value.
 - Rede Neural: A rede neural recebe o estado como entrada e produz os Q-values para todas as ações possíveis como saída. Inicialmente, os pesos da rede são aleatórios, então suas previsões são sem sentido. Esta abordagem é conhecida como Deep Q-Network (DQN).
- O estado S é uma representação da condição do ambiente no momento.
 - O estado deve conter todas as informações relevantes para que o agente possa tomar uma decisão.
- O agente precisa decidir qual ação (A) tomar com base em seu conhecimento atual.
 - Exploração (Greedy): Com uma probabilidade de $1-\epsilon$, o agente escolhe a ação que tem o maior Q-value (ou a maior saída da rede neural) para o estado atual.
 - Exploração: Com uma pequena probabilidade de ϵ , o agente escolhe uma ação aleatória. A exploração permite que o agente descubra novos caminhos e estratégias que ele ainda não conhece.
 - O valor de ϵ geralmente começa alto e diminui com o tempo, pois o agente precisa explorar mais no início e explora menos à medida que sua "memória" se torna mais confiável.
- O agente executa a ação escolhida no ambiente. Como resultado, o ambiente muda, e o agente recebe uma recompensa (R) imediata.
 - Recompensa Positiva / Recompensa Negativa / Recompensa Neutra
- Após a ação, o ambiente transiciona para um novo estado (S'). O agente observa esse novo estado para se preparar para o próximo ciclo de interação. Este novo estado é fundamental para a fase de atualização, pois ele contém a informação sobre o que aconteceu após a ação.
- O agente usa a informação da sua experiência recente (estado S , ação A , recompensa R e novo estado S') para atualizar seu conhecimento.
- Um episódio termina quando o agente atinge um estado final (como o objetivo ou uma falha) ou após um número máximo de passos. A repetição desse laço faz com que os valores da Q-table (ou os pesos da rede neural) encontrem os valores ótimos.

Aprendizado Por Reforço

- Policy Gradient

- Aprende diretamente uma função (política) que mapeia estados para probabilidades de ações, ajustando os pesos para maximizar recompensas esperadas.
- A política é a estratégia ou regra que o agente usa para decidir qual ação tomar em um determinado estado.
- O agente possui uma política, geralmente representada por uma rede neural.
- Essa rede pega o estado do ambiente como entrada e, como saída, dá a probabilidade de cada ação possível.
- O agente interage com o ambiente, toma ações com base nas probabilidades da sua política, e recebe recompensas.
- Ajustar os parâmetros da rede neural para que as ações que levaram a mais recompensas se tornem mais prováveis no futuro.
- O algoritmo calcula o gradiente da recompensa, que indica a "direção" em que os parâmetros da política devem ser modificados para aumentar a recompensa esperada.
- Em seguida, ele utiliza um otimizador (como o Gradient Ascent) para atualizar os pesos da rede neural, movendo a política nessa direção "melhor".

Hands-On

- No decorrer da aula vimos alguns exemplos de treinamento de máquina para regressão/predição e para classificação.
 - Vamos escolher dois métodos de regressão/predição **ou**
 - Dois métodos de classificação
 - Avaliar os parâmetros do treino dos dois
 - Imaginar o que resultará em hardware
 - Desenvolver em Verilog / VHDL
 - Vamos estimular os módulos e observar os resultados.
- Regressão Linear e Árvore de Decisão
- Regressão Logística e K-means

Página em branco.

Aula 14

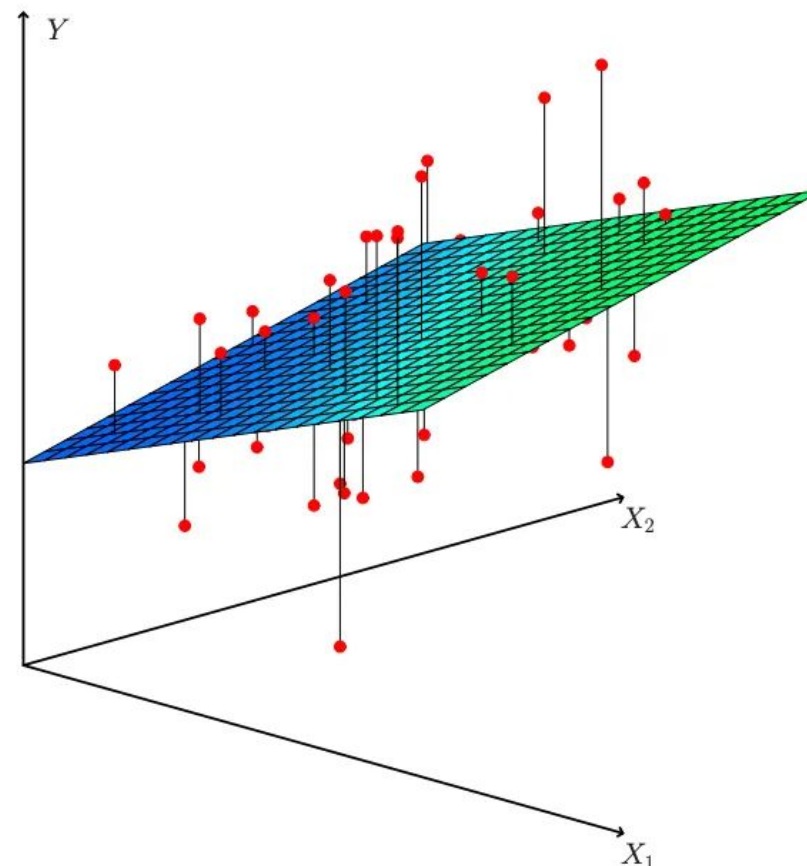
Arquiteturas para Machine Learning (ML) Regressão Linear Múltipla

Mapeamento de arquitetura ML: Regressão linear múltipla

Em aprendizagem de máquina, a regressão linear múltipla é uma técnica que permite realizar uma combinação linear entre diversas variáveis de entrada para obter uma saída. O modelo é o seguinte:

$$\hat{y}_k^L = c_1 \cdot x_{1,k} + c_2 \cdot x_{2,k} + \dots + c_{n_{var}} \cdot x_{n_{var},k} + b,$$

onde, c_i e b são números reais que representam os coeficientes do modelo, n_{var} é um número inteiro que representa o número de entradas, x_i e y representam as entradas e a saída, e k representa o tempo.



Mapeamento de arquitetura ML: Regressão linear múltipla

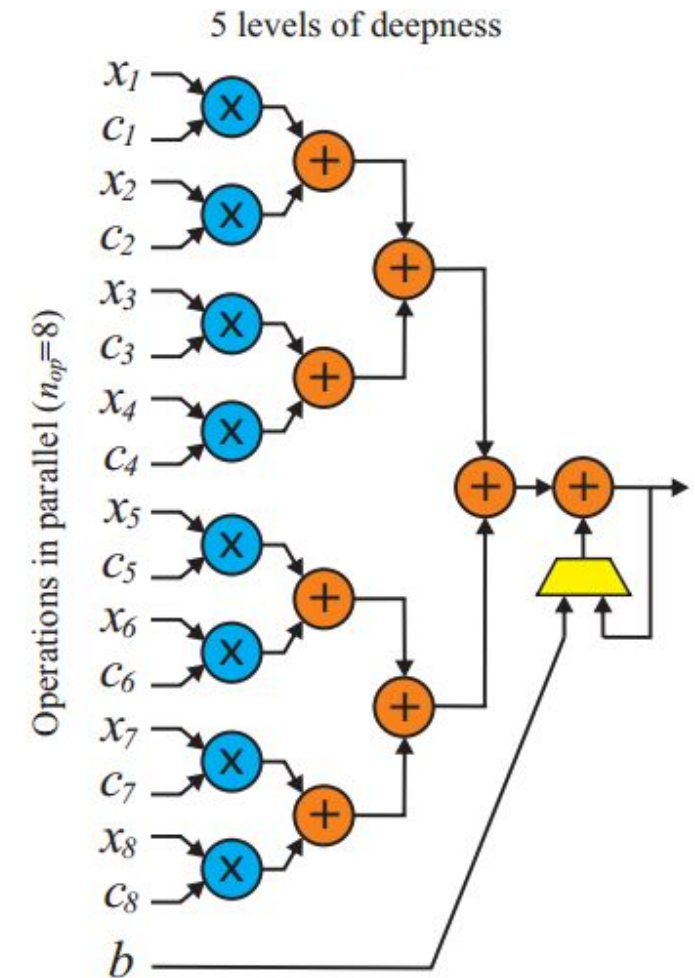
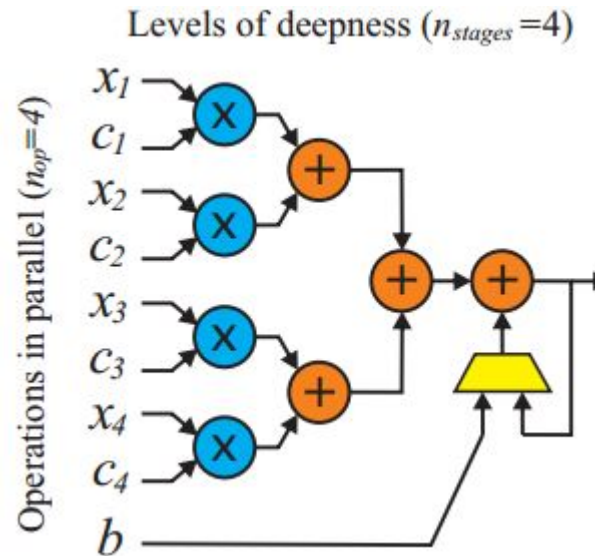
Exercício 1: Suponha um modelo com 64 entradas ($n_{var}=64$) provenientes das leituras de 64 sensores. Desenvolva uma arquitetura de hardware para a regressão linear múltipla supondo que os coeficientes c_i e b estão armazenados em memória. Suponha que as leituras dos sensores são de 32 bits e são fornecidas sequencialmente (uma entrada de 32 bits a cada ciclo de clock) por um sistema externo ao circuito.

Mapeamento de arquitetura ML: Regressão linear múltipla

Exercício 1: Regressão linear múltipla.

Observemos duas arquiteturas pipeline com diferente grau de paralelismo. A primeira usa quatro multiplicadores em paralelo e uma árvore de somas (4 estágios em total). A segunda usa oito multiplicadores e uma árvore de somas (5 estágios em total).

Nota: Suponha que a saída de cada operação está registrada.



Mapeamento de arquitetura ML: Regressão linear múltipla

Exercício 1: Qual é número ideal de multiplicadores em paralelo?

A resposta depende de vários fatores:

- Qual a interface e taxa dos dados de entrada? número de leituras por segundo.
- Qual a interface e taxa de dados de saída requerida?
- Qual a frequência de clock desejada?
- Quantos recursos de hardware temos disponíveis?
- Quantos multiplicadores temos disponíveis? Qual a latência de cálculo do multiplicador? e do somador?
- Qual o consumo de energia aceitável para a aplicação?

Vamos supor que a taxa de dados de entrada é de 1 amostra por ciclo de clock, e que o número máximo de multiplicadores é de 10 cuja latência é de dois clocks. Em consequência, espera-se um regressor que forneça uma saída a cada dois clocks. A frequência mínima do clock deve ser de 100 MHz.

Mapeamento de arquitetura ML: Regressão linear múltipla

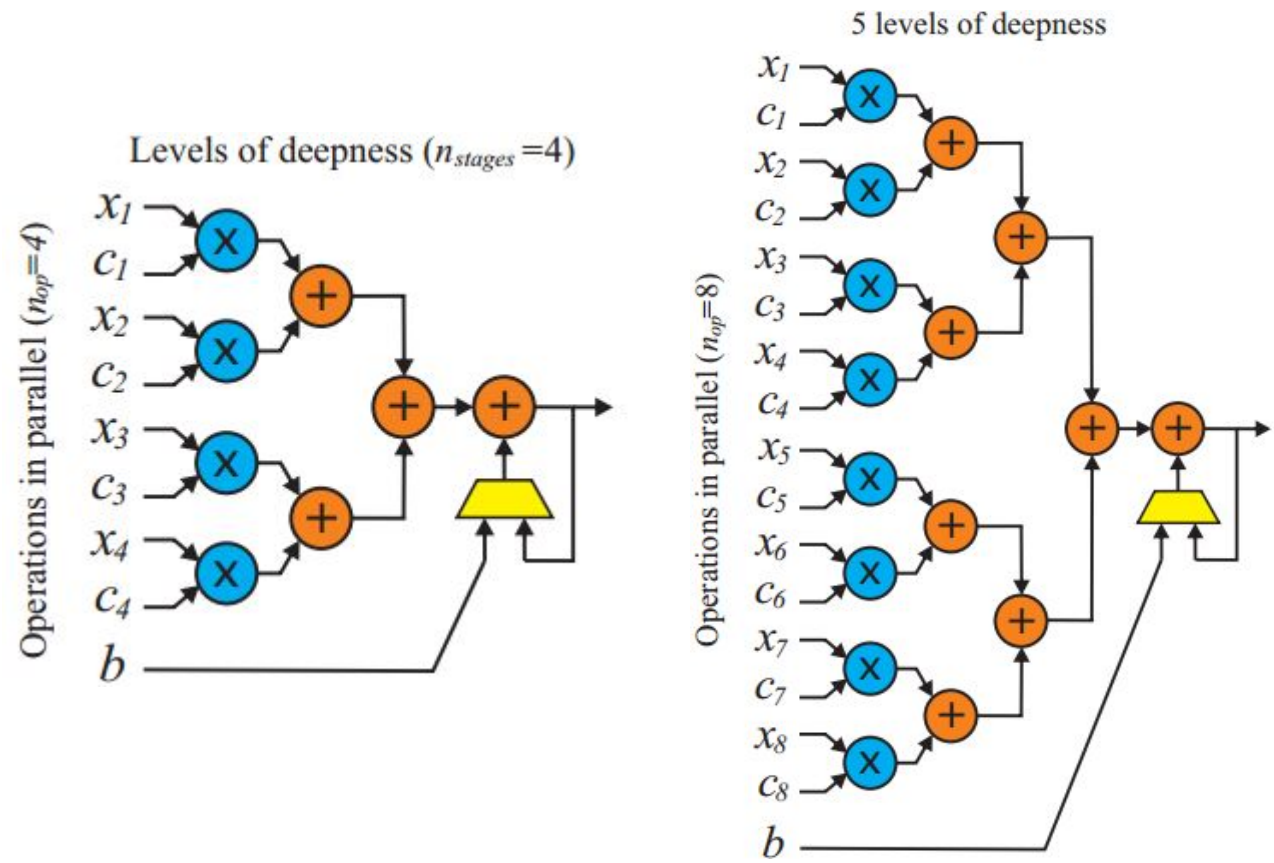
Exercício 1: Implemente uma arquitetura pipeline de um regressor linear de 64 entradas. As entradas são lidas sequencialmente. Uma saída de 32 bits.

- Número de entradas = 64
- Freq clock = 100 MHz
- Taxa de entradas: 1 cada clock
- Tamanho em bits entradas: 32
- Taxa saída: 1 cada dois clocks
- Tamanho em bits saída: 32
- Número máx de multiplicadores=10
- Latência multiplicador = 2 clocks
- Coeficientes c_i e b de 32 bits armazenados em memória ROM.

Sugestão 1: usar registrador de palavras SIPO para as entradas.

Sugestão 2: usar somadores e multiplicadores em ponto flutuante fornecidos pelo curso.

Sugestão 3: use o módulo de memória ROM fornecida pelo curso.

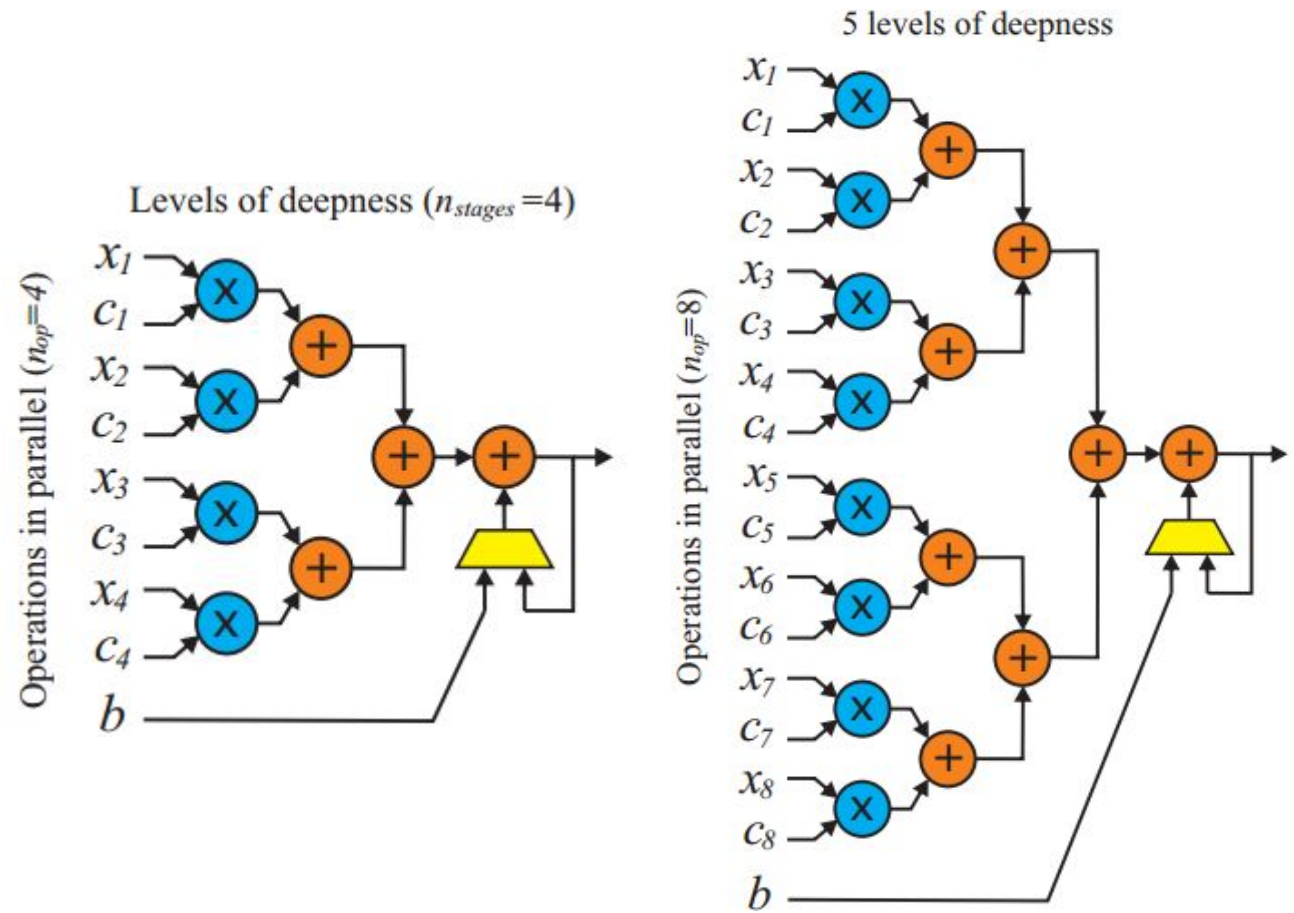


Mapeamento de arquitetura ML: Regressão linear múltipla

Exercício 1: Implemente uma arquitetura pipeline de um regressor linear de 64 entradas que atenda as restrições:

Etapas:

- Desenvolva o código RTL
- Realize a síntese lógica
- Obtenha o reporte de utilização de recursos.
- Desenvolva um testbench e verifique o funcionamento através de simulação comportamental
- Obtenha a latência e throughput da arquitetura.



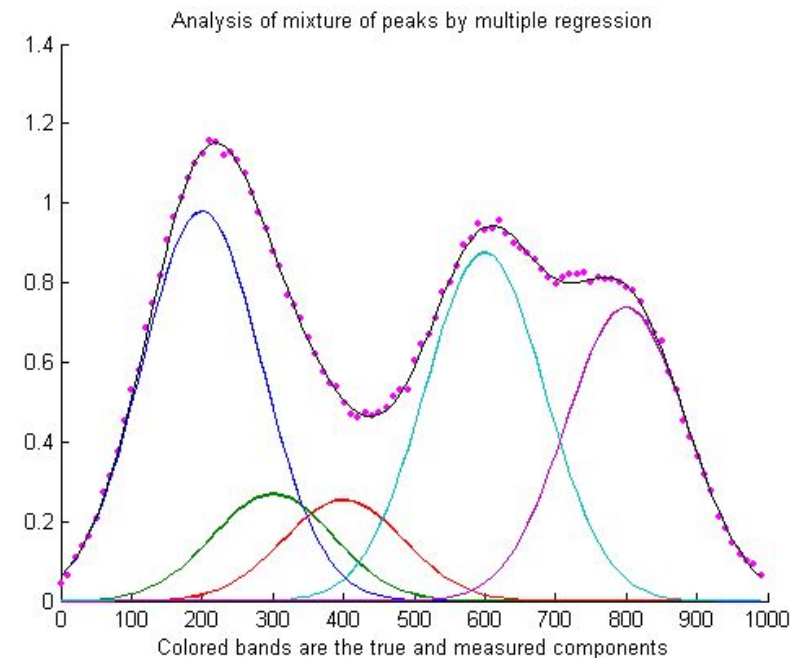
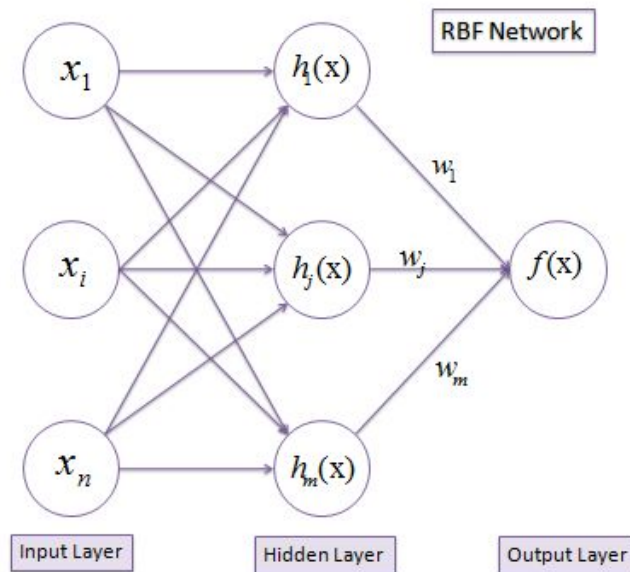
Página em branco.

Aula 15

Arquiteturas para Machine Learning (ML) Redes Neurais MLP e RBF

Mapeamento de arquitetura ML: redes RBF

Redes neurais de base radial (Radial Basis Function - RBF) são usadas para tarefas de regressão ou predição. Modelam os dados através de uma soma de gaussianas.

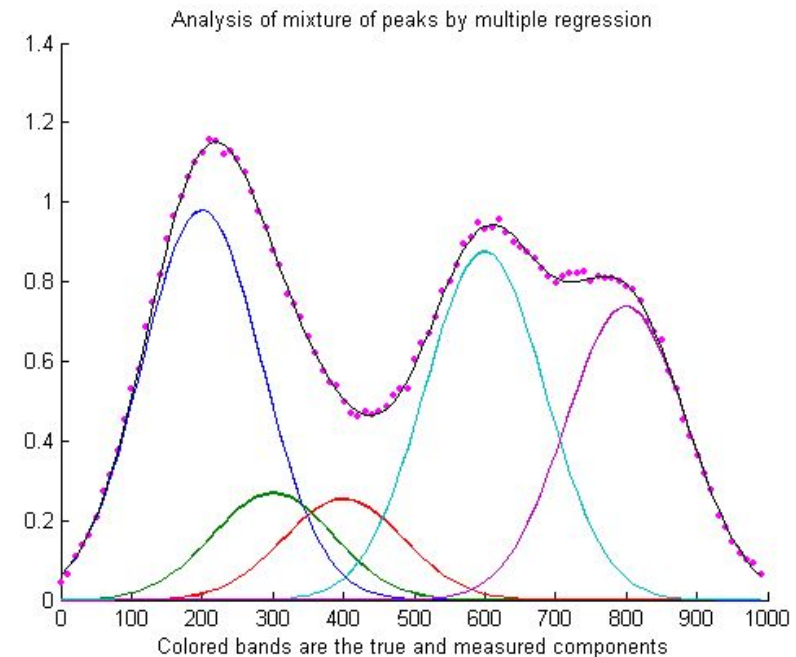
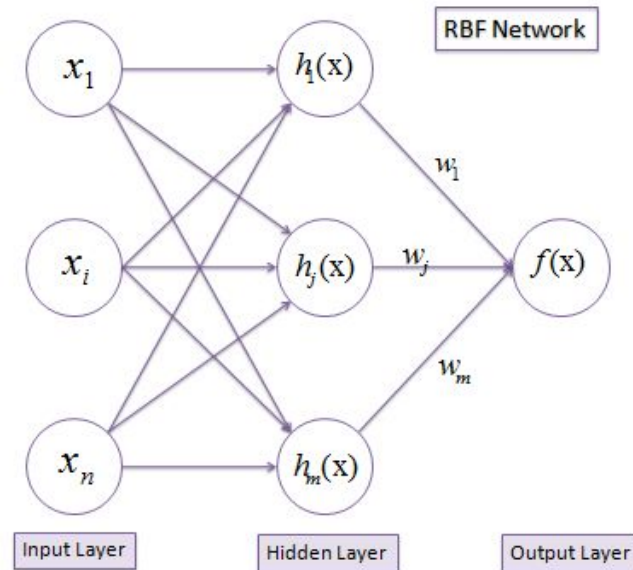


$$\phi(x(k), c_m, \sigma_m) = \exp\left(-\frac{1}{2\sigma_m^2} \sum_{i=1}^{n_{var}} (x_i(k) - c_{m,i})^2\right)$$

$$y_k = \sum_{m=1}^M w_m \phi(x_k, c_m, \sigma_m)$$

Mapeamento de arquitetura ML: redes RBF

Na camada intermediária cada neurônio possui um kernel Gaussiano com centro c e espalhamento r , obtidos por meio de treinamento supervisionado. Na camada de saída as gaussianas $h_j(x)$ são ponderadas por w_j e somadas para obter a função $f(x)$.



$$\phi(x(k), c_m, \sigma_m) = \exp\left(-\frac{1}{2\sigma_m^2} \sum_{i=1}^{n_{var}} (x_i(k) - c_{m,i})^2\right)$$

$$y_k = \sum_{m=1}^M w_m \phi(x_k, c_m, \sigma_m)$$

Mapeamento de arquitetura ML: redes RBF

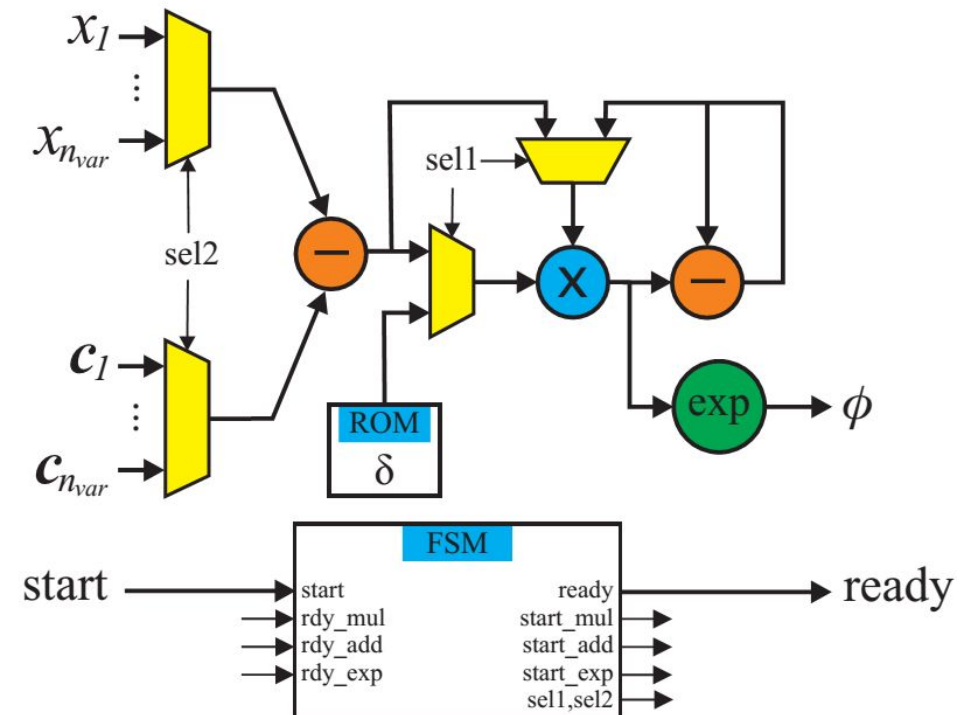
A seguinte arquitetura mapeia em hardware a expressão do neurônio RBF.

Arquitetura de controle de dados por FSM, que controla os operadores e multiplexadores. A função exponencial pode ser implementada usando o algoritmo Cordic.

- 1º estado: subtraí-se $x_i - c_m$
- 2º estado: calcula o quadrado de $(x_i - c_m)^2$
- 3º estado: acumula-se os desvios quadráticos
- 4º estado: multiplica-se por $\rho = 1/2\sigma^2$
- 5º estado: calcula-se a exponencial

Nota: A memória ROM armazena os valores de ρ e também poderia armazenar os valores dos centros das gaussianas.

$$\phi(x(k), c_m, \sigma_m) = \exp\left(-\frac{1}{2\sigma_m^2} \sum_{i=1}^{n_{var}} (x_i(k) - c_{m,i})^2\right)$$



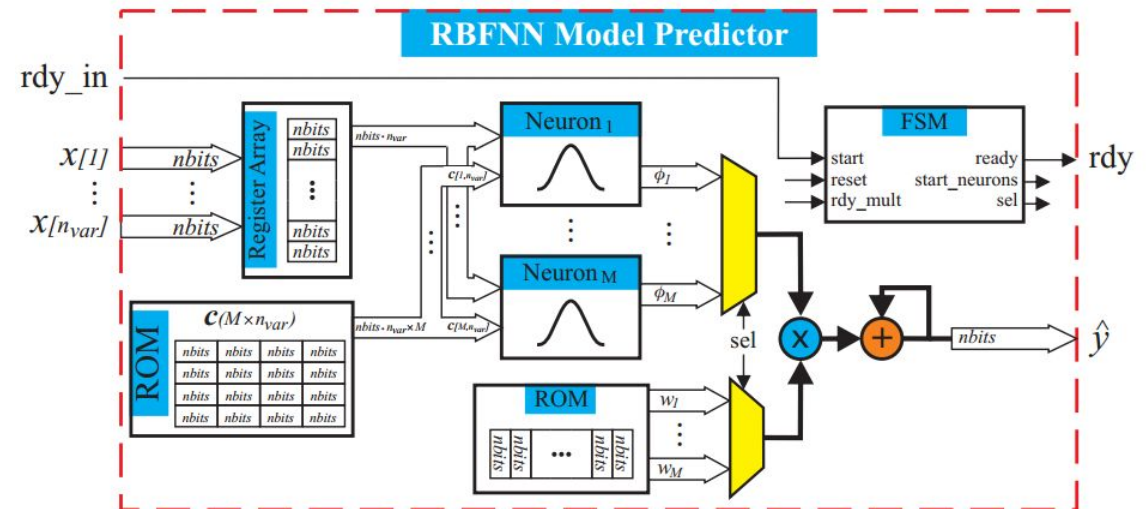
Mapeamento de arquitetura ML: redes RBF

A seguinte arquitetura mapeia implementa uma rede RBF usando neurônios em paralelo.

Arquitetura de controle de dados por FSM, que controla os operadores e multiplexadores.

- 1º estado: compõe array com as entradas.
- 2º estado: lê da ROM os valores dos centros dos neurônios e os direciona para cada neurônio.
- 3º estado: multiplica sequencialmente a saída de cada neurônio pelo respectivo peso w_m .
- 4º estado: acumula as saídas do multiplicador.

$$y_k = \sum_{m=1}^M w_m \phi(x_k, c_m, \sigma_m)$$



Mapeamento de arquitetura ML: Rede RBF

Exercício 2: Analise o código do neurônio RBF fornecido. Observe que os códigos não contêm os operadores de soma, subtração, multiplicação e da exponencial (Cordic).

Etapas:

- A. Desenvolva o código RTL incluindo os HDLs dos operadores de cálculo aritmético em ponto flutuante fornecidos pelo curso no módulo anterior. Deve-se respeitar o handshaking de start e ready de cada operador. O sinal start é um pulso de duração e um clock que indica que as entradas são válidas. O sinal ready é um pulso de saída com duração de um clock.
- B. Realize a síntese lógica
- C. Obtenha o reporte de utilização de recursos.
- D. Desenvolva um testbench e verifique o funcionamento através de simulação comportamental
- E. Obtenha a latência e throughput da arquitetura.

Página em branco.